Introductory Econometrics with Recitation: TA Session Week 10

Danny Po-Hsien Kang (康柏賢)

April 28, 2026

Today's agenda

1. Regular Expressions
   - `sub()`, `gsub()`
   - Regular Expressions
2. Regressions
   - `Polynomial Terms`
   - `geom_smooth()`
   - `Interaction Terms`
   - `predict()`

Today's Dataset

- Today we will use the following CSV file from the OpenIntro Website:
  - `acs12.csv`: Results from the US Census American Community Survey, 2012.
- Please import the following datasets into RStudio.

```
acs <- read.csv("data/acs12.csv")

acs <- acs %>%
  na.omit() %>%
  mutate(univ = ifelse(edu == "hs or lower", 0, 1)) %>%
  filter(income != 0)
```

String Replacement: sub() and gsub()

- We use sub() and gsub() to replace text inside a string.
- The difference is:
  - sub(): replace the **first match** only
  - gsub(): replace **all matches**
- **Syntax:**

```
sub(pattern, replacement, x)
gsub(pattern, replacement, x)
```

- pattern: the text or regular expression you want to find
- replacement: the new text
- x: the string or character vector

String Replacement: sub() and gsub()

- **Example:**

```
text <- "apple apple apple"

sub("apple", "pear", text)
gsub("apple", "pear", text)

gsub("-", "/", c("2025-04-22", "2025-04-29"))
```

- **Output:**

```
[1] "pear apple apple"
[1] "pear pear pear"

[1] "2025/04/22" "2025/04/29"
```

Regular Expressions: Basic

- A **regular expression** (regex) is a pattern used to match text.
- We often use regex with: `sub()`, `gsub()`, `grepl()`
- Regex lets us match a rule or pattern.
- **Common regex symbols:**

```
.       any single character
^       start of a string
$       end of a string
[abc]   one of a, b, c
[0-9]   one digit from 0 to 9
+       one or more times
*       zero or more times
```

- These symbols are very useful in cleaning text data.

Regular Expressions:^ and $

- **Example:**

```
words <- c("cat", "catalog", "mycat")

grepl("^cat", words)
grepl("cat$", words)
```

- **Output:**

```
[1]  TRUE  TRUE FALSE
[1]  TRUE FALSE  TRUE
```

- ^cat means the string starts with "cat".
- cat$ means the string ends with "cat".

Regular Expressions: [0-9]

- **Example:**

```
text <- c("room12", "apple", "A7")

grepl("[0-9]", text)
gsub("[0-9]", "", text)
```

- **Output:**

```
[1]  TRUE FALSE  TRUE
[1] "room"  "apple" "A"
```

- [0-9] matches any digit from 0 to 9.

Regular Expressions: .

- **Example:**

```
strings <- c("cat", "cot", "cut", "coat")

grepl("c.t", strings)
```

- **Output:**

```
[1]  TRUE  TRUE  TRUE FALSE
```

- . means any single character.
- So "c.t" matches "cat", "cot", and "cut".

Regular Expressions: String Cleaning

- **Example: remove unit labels**

```
weight <- c("52kg", "61kg", "48kg")

gsub("kg", "", weight)
```

- **Output:**

```
[1] "52" "61" "48"
```

- **Example: remove all digits**

```
gsub("[0-9]", "", weight)
```

- **Output:**

```
[1] "kg" "kg" "kg"
```

Regular Expressions: * and +

- **Meaning:**

```
*    zero or more times
+    one or more times
```

- ab* means:
  - "a"
  - "ab"
  - "abb"
  - "abbb", ...
- ab+ means:
  - "ab"
  - "abb"
  - "abbb", ...

Regular Expressions: * and +

- **Example:**

```
strings <- c("a", "ab", "abb", "abbb", "ac")

grepl("ab*", strings)
grepl("ab+", strings)
```

- **Output:**

```
[1] TRUE TRUE TRUE TRUE TRUE
[1] FALSE TRUE TRUE TRUE FALSE
```

- ab* matches "a" first, and also matches the first part of "ac".
- ab+ requires at least one "b", so it does not match "a" or "ac".

Regular Expressions: Combining Symbols

- We can combine regex symbols to match more flexible patterns.
- `.*` is often used to mean **"anything in between"** .
- **Example:**

```
emails <- c("alice@gmail.com", "bob@ntu.edu.tw")
sub("@.*", "", emails)
```

- **Output:**

```
[1] "alice" "bob"
```

Nonlinear Regression: Polynomial Terms

- **We consider the following model:**

$$\text{income}_i = \beta_0 + \beta_1 \text{age}_i + \beta_2 \text{age}_i^2 + \varepsilon_i$$

- The quadratic term is included in the model.
- How do we create the quadratic term in R?

```
acs <- acs %>%
  mutate(age2 = age * age)
```

- We can also use a built-in function to create polynomial terms when estimating a model.

Nonlinear Regression: Polynomial Terms

- **Syntax:**

```
lm_model_poly <- lm_robust(
  income ~ age + I(age^2),
  data = acs
)
```

- **Output:**

```
               Estimate  Std. Error   t value     Pr(>|t|)
(Intercept) -67741.602 19928.832   -3.399       0.000
age           4853.192  1163.161    4.172       0.000
I(age^2)       -47.235    14.972   -3.155       0.002
```

Visualizing Regression: geom_smooth()

- geom_smooth() adds a fitted regression line to a scatter plot.
- **Syntax:**

```
ggplot(acs, aes(x = age, y = income)) +
  geom_point() +
  geom_smooth(method = "lm", formula = ..., se = ...)
```

- method = "lm" tells R to fit a linear regression model.
  - default: formula = y ~ x
- geom_smooth() also plots the confidence interval.
  - se = FALSE removes the shaded confidence interval.

Visualizing Regression: Polynomial Fit

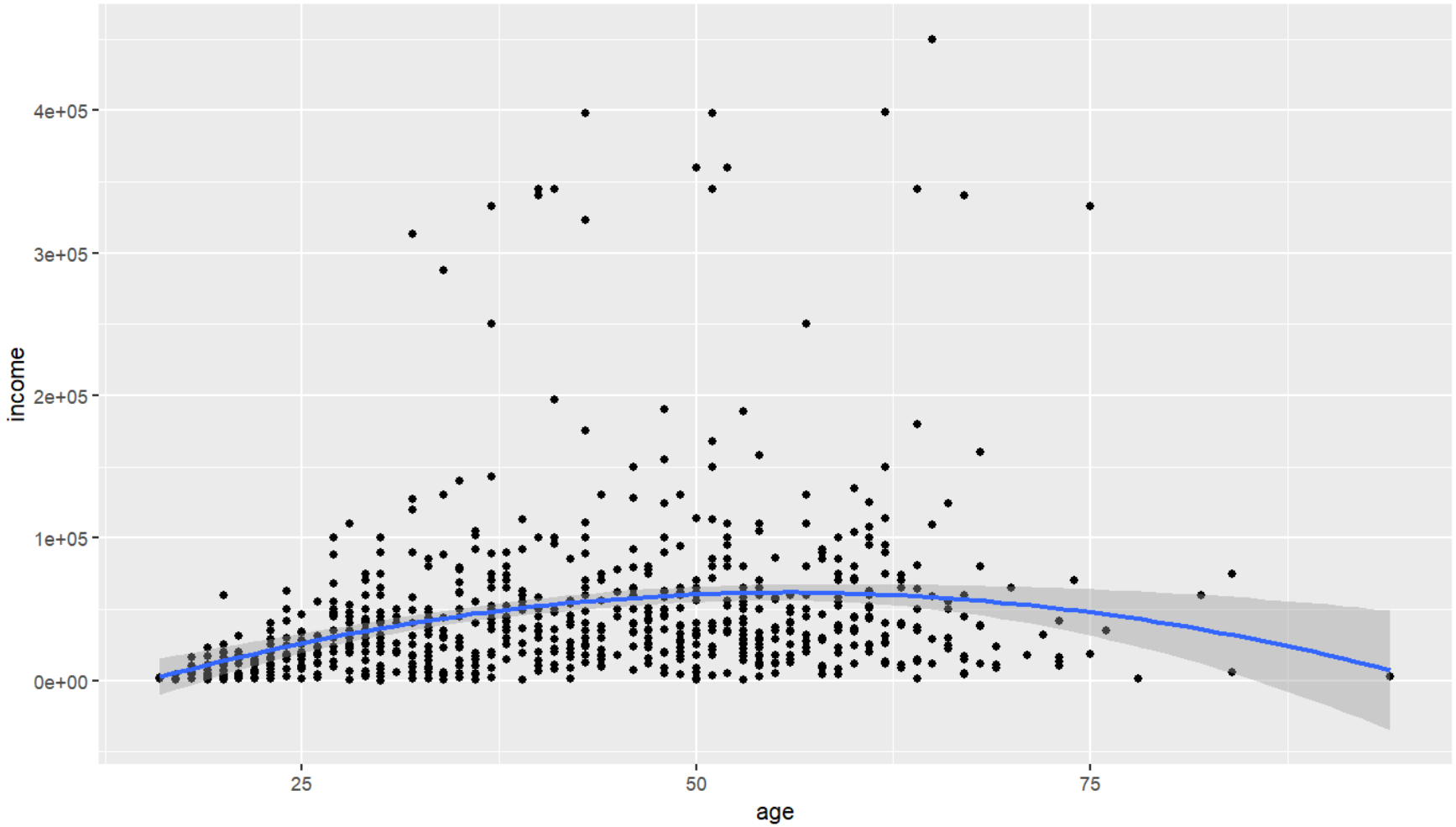
- We can visualize a polynomial regression model.
- **Example:**

```
ggplot(acs, aes(x = age, y = income)) +
  geom_point() +
  geom_smooth(
    method = "lm",
    formula = y ~ x + I(x^2)
  )
```

- This corresponds to the quadratic regression model:

$$\text{income}_i = \beta_0 + \beta_1 \text{age}_i + \beta_2 \text{age}_i^2 + \varepsilon_i$$

Visualizing Regression: Polynomial Fit



Nonlinear Regression: Interaction Terms

- **Syntax 1:**

```
lm_model_interaction <- lm_robust(
  income ~ age * univ,
  data = acs
)
```

- **Syntax 2:**

```
lm_model_interaction <- lm_robust(
  income ~ age + univ + age:univ,
  data = acs
)
```

- These two specifications are equivalent.

Nonlinear Regression: Interaction Terms

- **Output:**

	Estimate	Std. Error	t value	Pr(>|t|)
(Intercept)	10997.671	3589.542	3.064	0.002
age	505.137	82.779	6.102	0.000
univ	-2276.693	14040.242	-0.162	0.871
age:univ	913.643	357.976	2.552	0.011

Prediction: predict()

- **Syntax:**

```
predict(model, newdata = data_frame)
```

- Suppose there are two workers with different working hours:

```
# Initial working hour is 40hrs
low_hr <- data.frame(hrs_work = 40)

# Now increase by 2hrs to 42hrs
high_hr <- data.frame(hrs_work = 42)
```

Prediction: predict()

- First, we fit a model of income on hours worked.

```
model <- lm_robust(income ~ hrs_work, data = acs)
```

- Model output:

```
               Estimate Std. Error t value Pr(>|t|)
(Intercept) -16905.310   7511.361  -2.251    0.025
hrs_work      1641.663    209.689   7.829    0.000
```

- Income is predicted to increase by $2 \times 1641.663 = 3283.326$ dollars.

```
predict(model, high_hr) - predict(model, low_hr)
## 3283.326
```